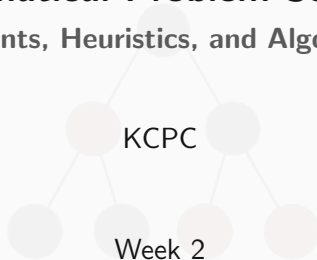
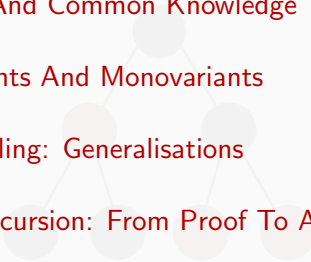


Mathematical Problem Solving II

Advanced Invariants, Heuristics, and Algorithmic Thinking



Overview

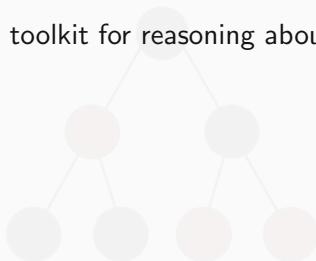
- 
- 1 Week 1 Recap and What Comes Next
 - 2 Advanced Logic And Common Knowledge
 - 3 Advanced Invariants And Monovariants
 - 4 Colouring And Tiling: Generalisations
 - 5 Induction And Recursion: From Proof To Algorithm
 - 6 Heuristics In Depth
 - 7 From Mathematical Insight To Algorithmic Thinking

Week 1 Recap and What Comes Next



Week 1 Recap

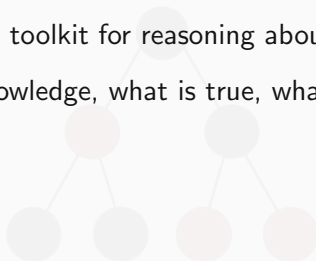
Last week we built a toolkit for reasoning about puzzles:



Week 1 Recap

Last week we built a toolkit for reasoning about puzzles:

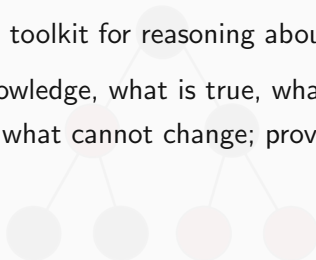
→ Logic: track knowledge, what is true, what is inferred.



Week 1 Recap

Last week we built a toolkit for reasoning about puzzles:

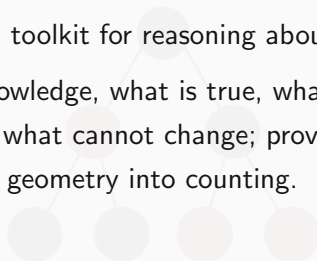
- Logic: track knowledge, what is true, what is inferred.
- Invariants: find what cannot change; prove impossibility.



Week 1 Recap

Last week we built a toolkit for reasoning about puzzles:

- Logic: track knowledge, what is true, what is inferred.
- Invariants: find what cannot change; prove impossibility.
- Colouring: turn geometry into counting.



Week 1 Recap

Last week we built a toolkit for reasoning about puzzles:

- Logic: track knowledge, what is true, what is inferred.
- Invariants: find what cannot change; prove impossibility.
- Colouring: turn geometry into counting.
- Induction: reduce the problem size step by step.

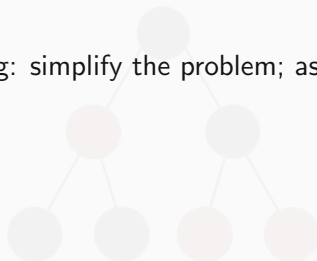
Week 1 Recap

Last week we built a toolkit for reasoning about puzzles:

- Logic: track knowledge, what is true, what is inferred.
- Invariants: find what cannot change; prove impossibility.
- Colouring: turn geometry into counting.
- Induction: reduce the problem size step by step.
- Monovariants: track progress towards a goal.

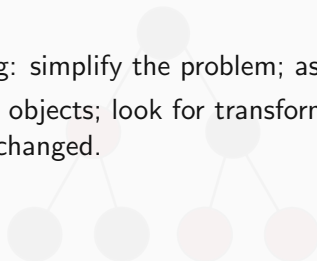
Heuristics: When You Are Stuck

→ Wishful thinking: simplify the problem; assume a nice form.

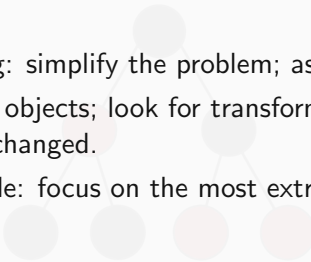


Heuristics: When You Are Stuck

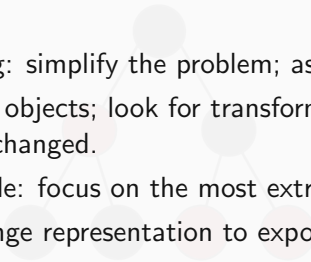
- Wishful thinking: simplify the problem; assume a nice form.
- Symmetry: pair objects; look for transformations that leave the problem unchanged.



Heuristics: When You Are Stuck

- 
- Wishful thinking: simplify the problem; assume a nice form.
 - Symmetry: pair objects; look for transformations that leave the problem unchanged.
 - Extreme principle: focus on the most extreme object.

Heuristics: When You Are Stuck

- 
- Wishful thinking: simplify the problem; assume a nice form.
 - Symmetry: pair objects; look for transformations that leave the problem unchanged.
 - Extreme principle: focus on the most extreme object.
 - Reduction: change representation to expose structure.

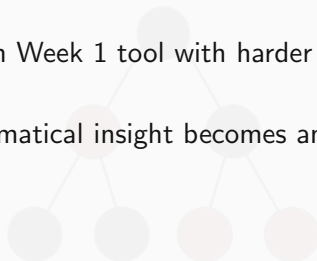
This Week: Deeper Tools And Algorithmic Bridges

- Strengthen each Week 1 tool with harder problems and generalisations.

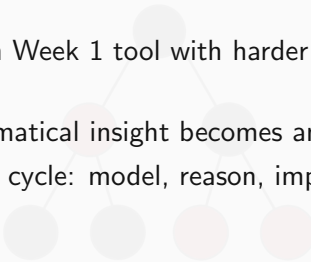


This Week: Deeper Tools And Algorithmic Bridges

- Strengthen each Week 1 tool with harder problems and generalisations.
- See how mathematical insight becomes an algorithmic idea.



This Week: Deeper Tools And Algorithmic Bridges

- 
- Strengthen each Week 1 tool with harder problems and generalisations.
 - See how mathematical insight becomes an algorithmic idea.
 - Practise the full cycle: model, reason, implement, verify.

This Week: Deeper Tools And Algorithmic Bridges

- 
- Strengthen each Week 1 tool with harder problems and generalisations.
 - See how mathematical insight becomes an algorithmic idea.
 - Practise the full cycle: model, reason, implement, verify.
 - Connect to standard CP platforms: CSES, Codeforces, AtCoder.

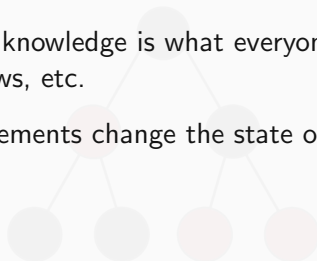
Advanced Logic And Common Knowledge



Logic Beyond Two Doors

Concept: Common knowledge is what everyone knows, everyone knows everyone knows, etc.

→ Public announcements change the state of knowledge.



Logic Beyond Two Doors

Concept: Common knowledge is what everyone knows, everyone knows everyone knows, etc.

- Public announcements change the state of knowledge.
- Reasoning depth: “I know that you know that I know...”

Logic Beyond Two Doors

Concept: Common knowledge is what everyone knows, everyone knows everyone knows, etc.

- Public announcements change the state of knowledge.
- Reasoning depth: “I know that you know that I know...”
- Often reduces to a fixed-point: when no new deductions are possible.

Worked Example: Muddy Children (Public Announcement)

n children play outside. k of them have mud on their foreheads. Each sees the others but not themselves.

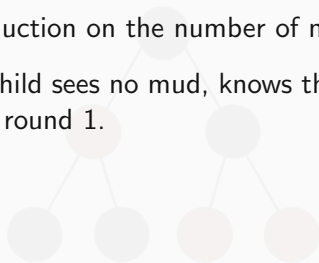
Father announces: "At least one of you has mud." Then he repeatedly asks: "Does anyone know their own state?"

What happens?

Muddy Children: Applying The Concept

How it applies: Induction on the number of muddy children k .

- If $k = 1$: that child sees no mud, knows they must be the one.
Answers yes on round 1.



Muddy Children: Applying The Concept

How it applies: Induction on the number of muddy children k .

- If $k = 1$: that child sees no mud, knows they must be the one. Answers yes on round 1.
- If $k = 2$: each sees one muddy child. If I were clean, the other would see 0 and answer on round 1. They don't. So I must be muddy. Both answer on round 2.

Muddy Children: Applying The Concept

How it applies: Induction on the number of muddy children k .

- If $k = 1$: that child sees no mud, knows they must be the one. Answers yes on round 1.
- If $k = 2$: each sees one muddy child. If I were clean, the other would see 0 and answer on round 1. They don't. So I must be muddy. Both answer on round 2.
- Induction: if k , all k answer on round k .

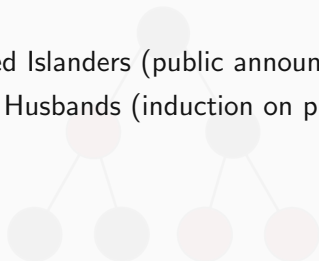
Student Exercises: Advanced Logic

→ Basic: Blue-Eyed Islanders (public announcement, $k = 2$).

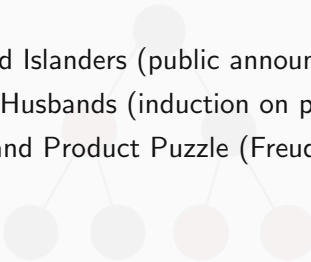


Student Exercises: Advanced Logic

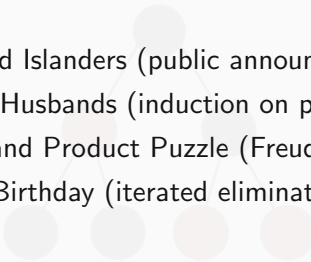
- Basic: Blue-Eyed Islanders (public announcement, $k = 2$).
- Easy: Cheating Husbands (induction on public knowledge).



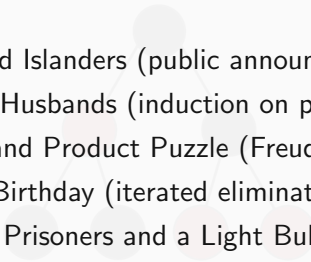
Student Exercises: Advanced Logic

- 
- Basic: Blue-Eyed Islanders (public announcement, $k = 2$).
 - Easy: Cheating Husbands (induction on public knowledge).
 - Medium: Sum and Product Puzzle (Freudenthal's problem).

Student Exercises: Advanced Logic

- 
- Basic: Blue-Eyed Islanders (public announcement, $k = 2$).
 - Easy: Cheating Husbands (induction on public knowledge).
 - Medium: Sum and Product Puzzle (Freudenthal's problem).
 - Hard: Cheryl's Birthday (iterated elimination).

Student Exercises: Advanced Logic

- 
- Basic: Blue-Eyed Islanders (public announcement, $k = 2$).
 - Easy: Cheating Husbands (induction on public knowledge).
 - Medium: Sum and Product Puzzle (Freudenthal's problem).
 - Hard: Cheryl's Birthday (iterated elimination).
 - Very Hard: 100 Prisoners and a Light Bulb (protocol design).

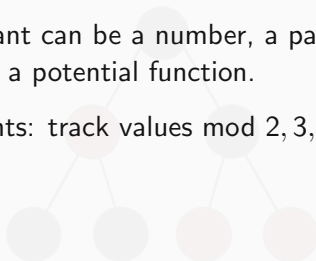
Advanced Invariants And Monovariants



Generalising Invariants

Concept: An invariant can be a number, a parity, a polynomial, a vector modulo m , or a potential function.

→ Modulo invariants: track values mod $2, 3, 4, \dots$



Generalising Invariants

Concept: An invariant can be a number, a parity, a polynomial, a vector modulo m , or a potential function.

- Modulo invariants: track values mod $2, 3, 4, \dots$
- Polynomial invariants: e.g., $x^2 + y^2$ for rotation.

Generalising Invariants

Concept: An invariant can be a number, a parity, a polynomial, a vector modulo m , or a potential function.

- Modulo invariants: track values mod $2, 3, 4, \dots$
- Polynomial invariants: e.g., $x^2 + y^2$ for rotation.
- Potential functions: sum of weights that never increases.

Generalising Invariants

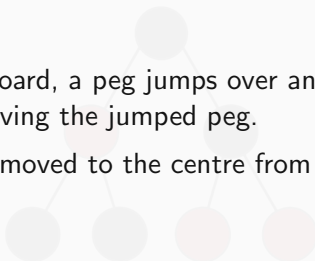
Concept: An invariant can be a number, a parity, a polynomial, a vector modulo m , or a potential function.

- Modulo invariants: track values mod $2, 3, 4, \dots$
- Polynomial invariants: e.g., $x^2 + y^2$ for rotation.
- Potential functions: sum of weights that never increases.
- Multiple invariants together can prove stronger results.

Worked Example: Peg Solitaire

On a cross-shaped board, a peg jumps over an adjacent peg into an empty hole, removing the jumped peg.

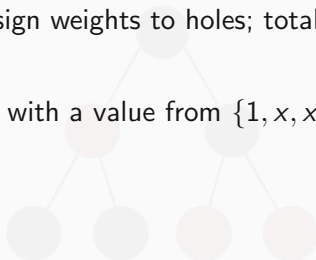
Can a single peg be moved to the centre from the standard starting position?



Peg Solitaire: Applying The Concept

How it applies: Assign weights to holes; total weight never increases.

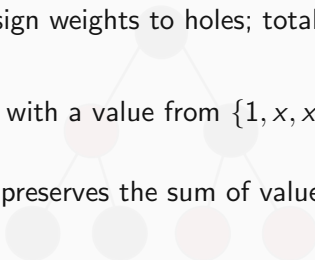
→ Label each hole with a value from $\{1, x, x^2\}$ where x satisfies $x^2 + x + 1 = 0$.



Peg Solitaire: Applying The Concept

How it applies: Assign weights to holes; total weight never increases.

- Label each hole with a value from $\{1, x, x^2\}$ where x satisfies $x^2 + x + 1 = 0$.
- Any legal jump preserves the sum of values of occupied holes.



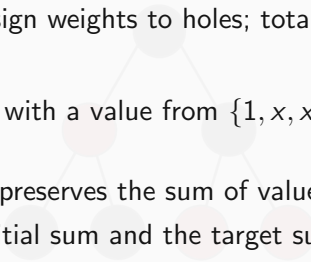
Peg Solitaire: Applying The Concept

How it applies: Assign weights to holes; total weight never increases.

- Label each hole with a value from $\{1, x, x^2\}$ where x satisfies $x^2 + x + 1 = 0$.
- Any legal jump preserves the sum of values of occupied holes.
- Compute the initial sum and the target sum.

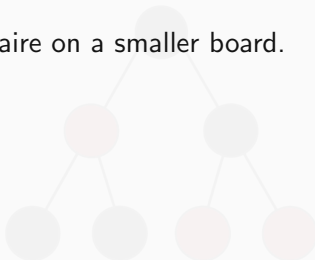
Peg Solitaire: Applying The Concept

How it applies: Assign weights to holes; total weight never increases.

- 
- Label each hole with a value from $\{1, x, x^2\}$ where x satisfies $x^2 + x + 1 = 0$.
 - Any legal jump preserves the sum of values of occupied holes.
 - Compute the initial sum and the target sum.
 - They differ, so the target is unreachable.

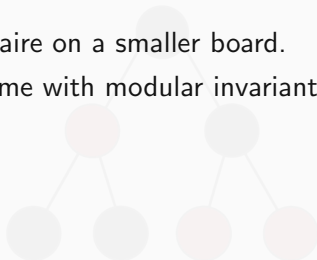
Student Exercises: Advanced Invariants

→ Basic: Peg solitaire on a smaller board.



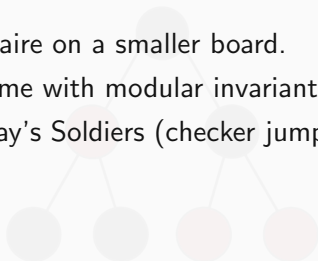
Student Exercises: Advanced Invariants

- Basic: Peg solitaire on a smaller board.
- Easy: Token game with modular invariant.

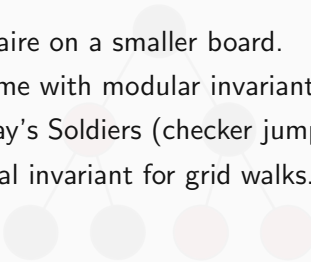


Student Exercises: Advanced Invariants

- Basic: Peg solitaire on a smaller board.
- Easy: Token game with modular invariant.
- Medium: Conway's Soldiers (checker jumping).



Student Exercises: Advanced Invariants

- 
- Basic: Peg solitaire on a smaller board.
 - Easy: Token game with modular invariant.
 - Medium: Conway's Soldiers (checker jumping).
 - Hard: Polynomial invariant for grid walks.

Student Exercises: Advanced Invariants

- Basic: Peg solitaire on a smaller board.
- Easy: Token game with modular invariant.
- Medium: Conway's Soldiers (checker jumping).
- Hard: Polynomial invariant for grid walks.
- Very Hard: Invariant for the IMO 1986 Problem 3 (integer sequence).

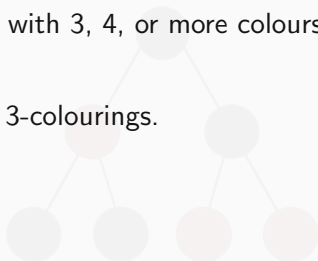
Colouring And Tiling: Generalisations



Beyond Two Colours

Concept: Colouring with 3, 4, or more colours captures more structure.

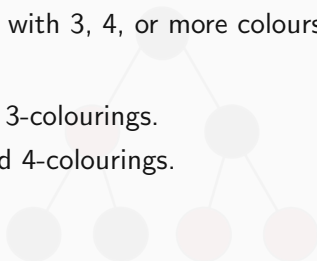
→ Triominoes and 3-colourings.



Beyond Two Colours

Concept: Colouring with 3, 4, or more colours captures more structure.

- Triominoes and 3-colourings.
- Tetrominoes and 4-colourings.



Beyond Two Colours

Concept: Colouring with 3, 4, or more colours captures more structure.

- Triominoes and 3-colourings.
- Tetrominoes and 4-colourings.
- Modular colourings: colour (i, j) by $(i + j) \bmod m$ or $(i + 2j) \bmod m$.

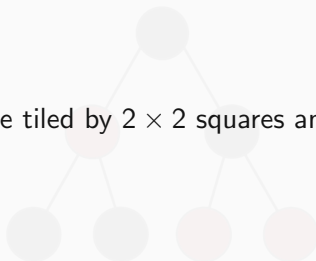
Beyond Two Colours

Concept: Colouring with 3, 4, or more colours captures more structure.

- Triominoes and 3-colourings.
- Tetrominoes and 4-colourings.
- Modular colourings: colour (i, j) by $(i + j) \bmod m$ or $(i + 2j) \bmod m$.
- Each tile shape interacts with colours in a predictable way.

Worked Example: Tiling With Trominoes

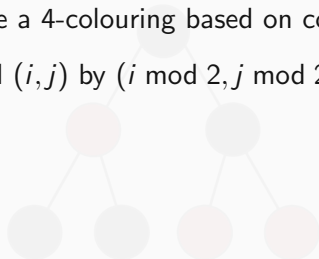
Can a 5×8 board be tiled by 2×2 squares and L-shaped trominoes?



Tromino Tiling: Applying The Concept

How it applies: Use a 4-colouring based on coordinates mod 2.

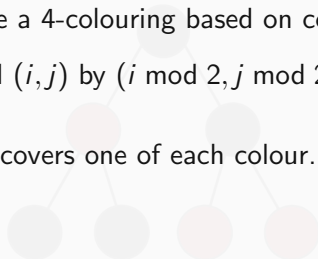
→ Colour each cell (i,j) by $(i \bmod 2, j \bmod 2)$: four colour classes.



Tromino Tiling: Applying The Concept

How it applies: Use a 4-colouring based on coordinates mod 2.

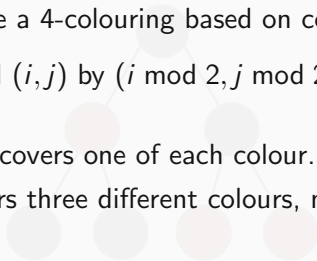
- Colour each cell (i, j) by $(i \bmod 2, j \bmod 2)$: four colour classes.
- A 2×2 square covers one of each colour.



Tromino Tiling: Applying The Concept

How it applies: Use a 4-colouring based on coordinates mod 2.

- Colour each cell (i,j) by $(i \bmod 2, j \bmod 2)$: four colour classes.
- A 2×2 square covers one of each colour.
- A tromino covers three different colours, missing one.



Tromino Tiling: Applying The Concept

How it applies: Use a 4-colouring based on coordinates mod 2.

- Colour each cell (i,j) by $(i \bmod 2, j \bmod 2)$: four colour classes.
- A 2×2 square covers one of each colour.
- A tromino covers three different colours, missing one.
- Count how many cells of each colour the board has.

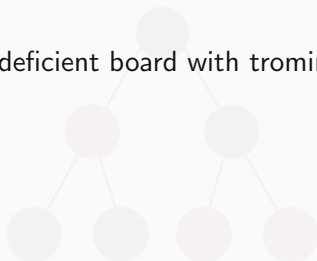
Tromino Tiling: Applying The Concept

How it applies: Use a 4-colouring based on coordinates mod 2.

- Colour each cell (i,j) by $(i \bmod 2, j \bmod 2)$: four colour classes.
- A 2×2 square covers one of each colour.
- A tromino covers three different colours, missing one.
- Count how many cells of each colour the board has.
- The counts show tiling is impossible.

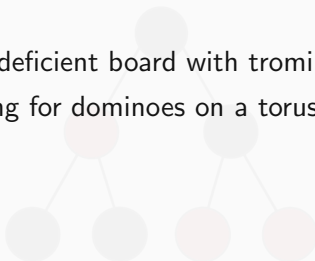
Student Exercises: Advanced Colouring

→ Basic: Tiling a deficient board with trominoes.



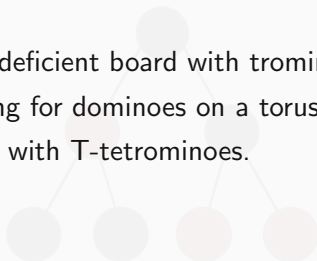
Student Exercises: Advanced Colouring

- Basic: Tiling a deficient board with trominoes.
- Easy: 3-colouring for dominoes on a torus.



Student Exercises: Advanced Colouring

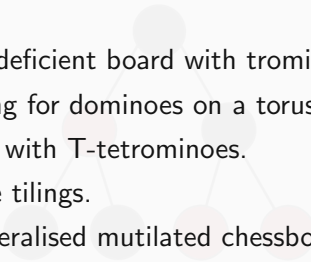
- Basic: Tiling a deficient board with trominoes.
- Easy: 3-colouring for dominoes on a torus.
- Medium: Tiling with T-tetrominoes.



Student Exercises: Advanced Colouring

- Basic: Tiling a deficient board with trominoes.
- Easy: 3-colouring for dominoes on a torus.
- Medium: Tiling with T-tetrominoes.
- Hard: Fault-free tilings.

Student Exercises: Advanced Colouring

- 
- Basic: Tiling a deficient board with trominoes.
 - Easy: 3-colouring for dominoes on a torus.
 - Medium: Tiling with T-tetrominoes.
 - Hard: Fault-free tilings.
 - Very Hard: Generalised mutilated chessboard.

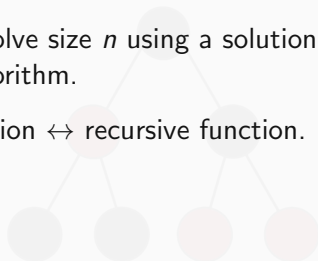
Induction And Recursion: From Proof To Algorithm



Induction As A Design Principle

Concept: If I can solve size n using a solution for size $n - 1$, I have a recursive algorithm.

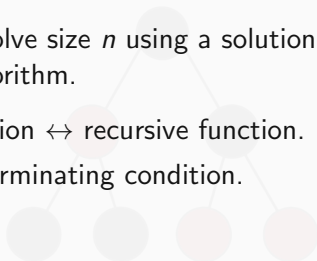
→ Proof by induction $\leftrightarrow$ recursive function.



Induction As A Design Principle

Concept: If I can solve size n using a solution for size $n - 1$, I have a recursive algorithm.

- Proof by induction $\leftrightarrow$ recursive function.
- Base case $\leftrightarrow$ terminating condition.



Induction As A Design Principle

Concept: If I can solve size n using a solution for size $n - 1$, I have a recursive algorithm.

- Proof by induction $\leftrightarrow$ recursive function.
- Base case $\leftrightarrow$ terminating condition.
- Inductive step $\leftrightarrow$ recursive call and combination.

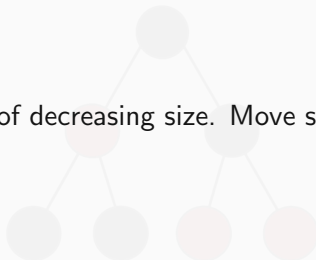
Induction As A Design Principle

Concept: If I can solve size n using a solution for size $n - 1$, I have a recursive algorithm.

- Proof by induction $\leftrightarrow$ recursive function.
- Base case $\leftrightarrow$ terminating condition.
- Inductive step $\leftrightarrow$ recursive call and combination.
- Strong induction $\leftrightarrow$ divide-and-conquer or DP.

Worked Example: Tower Of Hanoi

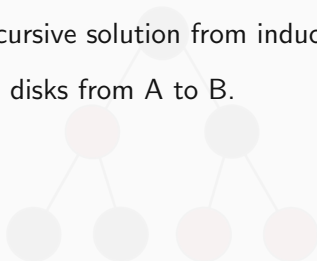
Three pegs, n disks of decreasing size. Move stack from peg A to peg C.



Tower Of Hanoi: Applying The Concept

How it applies: Recursive solution from induction on n .

→ Move top $n - 1$ disks from A to B.



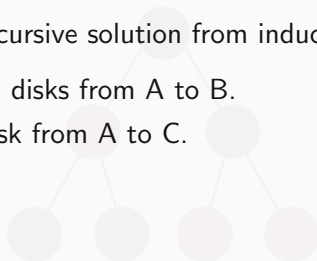
The recursive structure *is* the algorithm.

Tower Of Hanoi: Applying The Concept

How it applies: Recursive solution from induction on n .

→ Move top $n - 1$ disks from A to B.

→ Move largest disk from A to C.



The recursive structure *is* the algorithm.

Tower Of Hanoi: Applying The Concept

How it applies: Recursive solution from induction on n .

- Move top $n - 1$ disks from A to B.
- Move largest disk from A to C.
- Move $n - 1$ disks from B to C.

The recursive structure *is* the algorithm.

Tower Of Hanoi: Applying The Concept

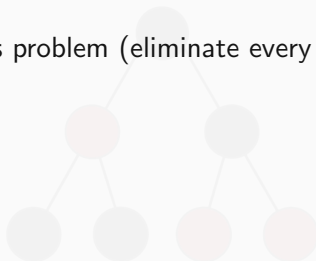
How it applies: Recursive solution from induction on n .

- Move top $n - 1$ disks from A to B.
- Move largest disk from A to C.
- Move $n - 1$ disks from B to C.
- Minimum moves: $2^n - 1$ (prove by induction).

The recursive structure *is* the algorithm.

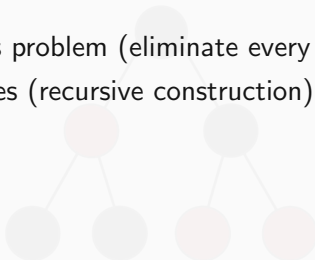
Student Exercises: Induction To Algorithm

→ Basic: Josephus problem (eliminate every second person).



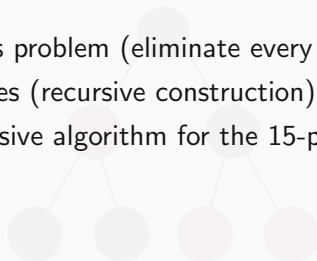
Student Exercises: Induction To Algorithm

- Basic: Josephus problem (eliminate every second person).
- Easy: Gray codes (recursive construction).



Student Exercises: Induction To Algorithm

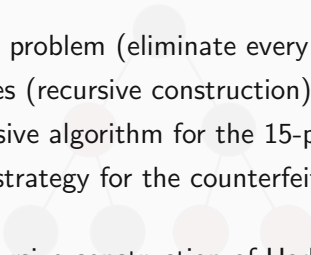
- Basic: Josephus problem (eliminate every second person).
- Easy: Gray codes (recursive construction).
- Medium: Recursive algorithm for the 15-puzzle parity.



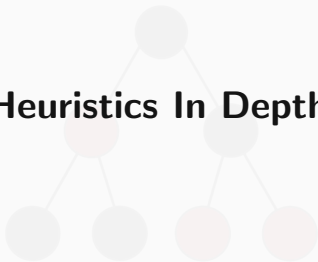
Student Exercises: Induction To Algorithm

- Basic: Josephus problem (eliminate every second person).
- Easy: Gray codes (recursive construction).
- Medium: Recursive algorithm for the 15-puzzle parity.
- Hard: Optimal strategy for the counterfeit coin with n weighings.

Student Exercises: Induction To Algorithm

- 
- Basic: Josephus problem (eliminate every second person).
 - Easy: Gray codes (recursive construction).
 - Medium: Recursive algorithm for the 15-puzzle parity.
 - Hard: Optimal strategy for the counterfeit coin with n weighings.
 - Very Hard: Recursive construction of Hadamard matrices.

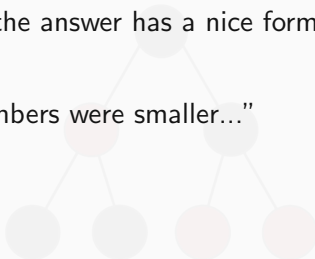
Heuristics In Depth



Wishful Thinking: Simplify Until Solvable

Heuristic: Assume the answer has a nice form. What would make this easy?

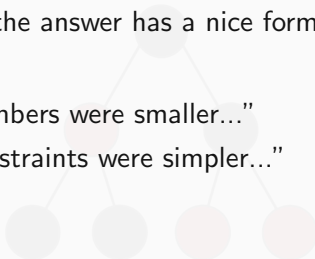
→ “If only the numbers were smaller...”



Wishful Thinking: Simplify Until Solvable

Heuristic: Assume the answer has a nice form. What would make this easy?

- “If only the numbers were smaller...”
- “If only the constraints were simpler...”



Wishful Thinking: Simplify Until Solvable

Heuristic: Assume the answer has a nice form. What would make this easy?

- “If only the numbers were smaller...”
- “If only the constraints were simpler...”
- “What would the solution look like if I already knew it?”

Wishful Thinking: Simplify Until Solvable

Heuristic: Assume the answer has a nice form. What would make this easy?

- “If only the numbers were smaller...”
- “If only the constraints were simpler...”
- “What would the solution look like if I already knew it?”
- Then find the reduction from the real problem to the simplified one.

Worked Example: Wishful Thinking – Sum of Squares

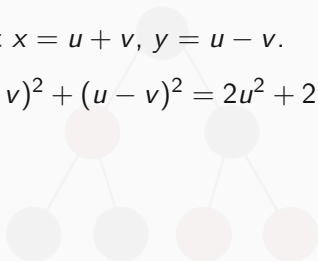
Find all integer solutions to $x^2 + y^2 = 2z^2$ with $x, y, z > 0$.

Wishful thinking: what if $x + y$ and $x - y$ had a nice structure?

Wishful Thinking: Applying The Concept

How it applies: Set $x = u + v$, $y = u - v$.

$$\rightarrow x^2 + y^2 = (u + v)^2 + (u - v)^2 = 2u^2 + 2v^2 = 2z^2.$$



Wishful Thinking: Applying The Concept

How it applies: Set $x = u + v$, $y = u - v$.

→ $x^2 + y^2 = (u + v)^2 + (u - v)^2 = 2u^2 + 2v^2 = 2z^2$.

→ So $u^2 + v^2 = z^2$ – a Pythagorean triple!

Wishful Thinking: Applying The Concept

How it applies: Set $x = u + v$, $y = u - v$.

→ $x^2 + y^2 = (u + v)^2 + (u - v)^2 = 2u^2 + 2v^2 = 2z^2$.

→ So $u^2 + v^2 = z^2$ – a Pythagorean triple!

→ All solutions: $u = m^2 - n^2$, $v = 2mn$, $z = m^2 + n^2$ (or swapped).

Wishful Thinking: Applying The Concept

How it applies: Set $x = u + v$, $y = u - v$.

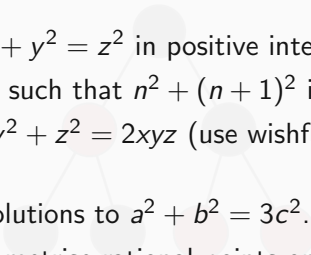
→ $x^2 + y^2 = (u + v)^2 + (u - v)^2 = 2u^2 + 2v^2 = 2z^2$.

→ So $u^2 + v^2 = z^2$ – a Pythagorean triple!

→ All solutions: $u = m^2 - n^2$, $v = 2mn$, $z = m^2 + n^2$ (or swapped).

→ Then $x = (m^2 - n^2) + 2mn$, $y = (m^2 - n^2) - 2mn$.

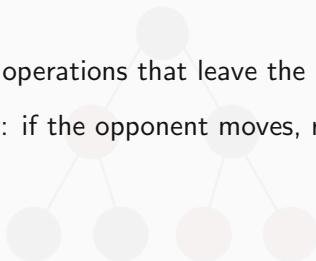
Student Exercises: Wishful Thinking

- 
- Basic: Solve $x^2 + y^2 = z^2$ in positive integers.
 - Easy: Find all n such that $n^2 + (n + 1)^2$ is a perfect square.
 - Medium: $x^2 + y^2 + z^2 = 2xyz$ (use wishful thinking to reduce).
 - Hard: Integer solutions to $a^2 + b^2 = 3c^2$.
 - Very Hard: Parametrise rational points on $x^3 + y^3 = 9$.

Symmetry: Exploit Invariance Under Transformations

Heuristic: Look for operations that leave the problem unchanged.

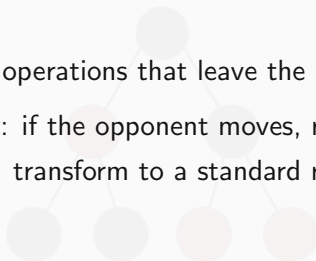
→ Pairing strategy: if the opponent moves, reply symmetrically.



Symmetry: Exploit Invariance Under Transformations

Heuristic: Look for operations that leave the problem unchanged.

- Pairing strategy: if the opponent moves, reply symmetrically.
- Canonical form: transform to a standard representative.



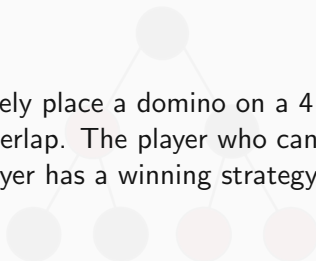
Symmetry: Exploit Invariance Under Transformations

Heuristic: Look for operations that leave the problem unchanged.

- Pairing strategy: if the opponent moves, reply symmetrically.
- Canonical form: transform to a standard representative.
- Symmetric positions often have symmetric optimal moves.

Worked Example: Symmetry – Pairing Game

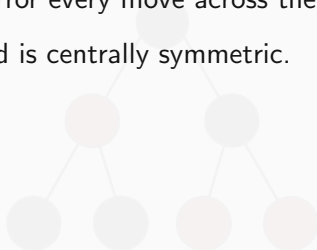
Two players alternately place a domino on a 4×4 board. Dominoes cannot overlap. The player who cannot move loses. Show the second player has a winning strategy.



Symmetry: Applying The Concept

How it applies: Mirror every move across the centre.

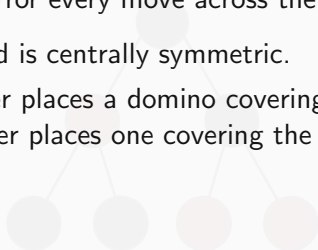
→ The 4×4 board is centrally symmetric.



Symmetry: Applying The Concept

How it applies: Mirror every move across the centre.

- The 4×4 board is centrally symmetric.
- If the first player places a domino covering (x, y) and (x', y') , the second player places one covering the centrally symmetric squares.



Symmetry: Applying The Concept

How it applies: Mirror every move across the centre.

- The 4×4 board is centrally symmetric.
- If the first player places a domino covering (x, y) and (x', y') , the second player places one covering the centrally symmetric squares.
- This move is always legal because the board was symmetric and the first player's move was legal.

Symmetry: Applying The Concept

How it applies: Mirror every move across the centre.

- The 4×4 board is centrally symmetric.
- If the first player places a domino covering (x, y) and (x', y') , the second player places one covering the centrally symmetric squares.
- This move is always legal because the board was symmetric and the first player's move was legal.
- The second player never runs out of moves first, so wins.

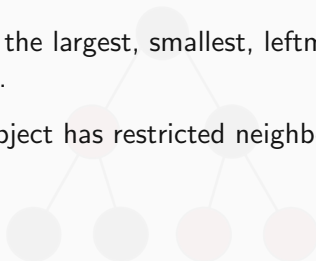
Student Exercises: Symmetry

- Basic: Pairing game on a $2 \times n$ board.
- Easy: Symmetry strategy in Chomp (poisoned chocolate).
- Medium: Second player wins on even $\times$ even board with centrally symmetric tiles.
- Hard: Symmetry in the game of Nim with paired heaps.
- Very Hard: Strategy stealing argument for Hex.

Extreme Principle: The Boundary Holds The Answer

Heuristic: Consider the largest, smallest, leftmost, rightmost, or most extreme object.

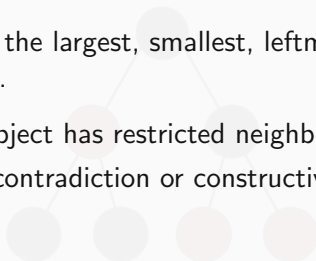
→ The extremal object has restricted neighbours.



Extreme Principle: The Boundary Holds The Answer

Heuristic: Consider the largest, smallest, leftmost, rightmost, or most extreme object.

- The extremal object has restricted neighbours.
- Often leads to contradiction or constructive greedy choice.



Extreme Principle: The Boundary Holds The Answer

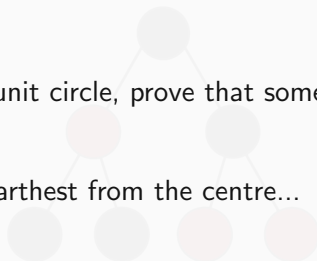
Heuristic: Consider the largest, smallest, leftmost, rightmost, or most extreme object.

- The extremal object has restricted neighbours.
- Often leads to contradiction or constructive greedy choice.
- Works in geometry, combinatorics, number theory, and algorithms.

Worked Example: Extreme Principle – Points in a Circle

Given 5 points in a unit circle, prove that some pair is at distance ≤ 1 .

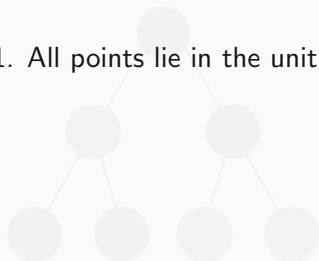
Consider the point farthest from the centre...



Extreme Principle: Applying The Concept

How it applies: Let P be the point with maximum distance from the centre O .

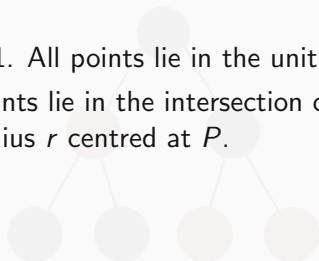
→ Let $OP = r \leq 1$. All points lie in the unit circle centred at O .



Extreme Principle: Applying The Concept

How it applies: Let P be the point with maximum distance from the centre O .

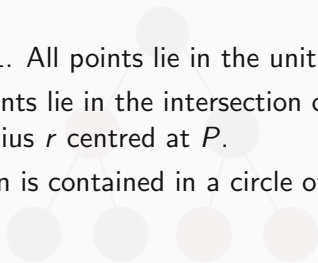
- Let $OP = r \leq 1$. All points lie in the unit circle centred at O .
- The other 4 points lie in the intersection of the unit circle and the circle of radius r centred at P .



Extreme Principle: Applying The Concept

How it applies: Let P be the point with maximum distance from the centre O .

- Let $OP = r \leq 1$. All points lie in the unit circle centred at O .
- The other 4 points lie in the intersection of the unit circle and the circle of radius r centred at P .
- This intersection is contained in a circle of radius $r/2$...



Extreme Principle: Applying The Concept

How it applies: Let P be the point with maximum distance from the centre O .

- Let $OP = r \leq 1$. All points lie in the unit circle centred at O .
- The other 4 points lie in the intersection of the unit circle and the circle of radius r centred at P .
- This intersection is contained in a circle of radius $r/2$...
- (Standard proof: divide into 5 sectors of 72° , or use the fact that the maximum minimum distance for 5 points in a unit circle is $2 \sin(36^\circ) > 1$; the claim as stated needs unit circle of diameter 1. Adjusting: for 5 points in a circle of diameter 1, some pair is at distance $\leq 1/2$.)

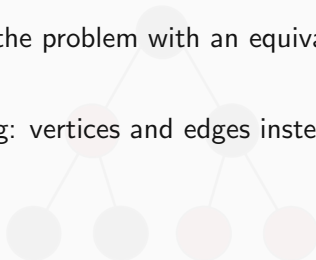
Student Exercises: Extreme Principle

- Basic: 5 points in a unit square; some pair at distance $\leq \sqrt{2}/2$.
- Easy: In any set of 10 integers, some subset sums to a multiple of 10 (consider extreme partial sums).
- Medium: Maximum area triangle inscribed in a circle is equilateral (extreme principle on area).
- Hard: In a tournament, there is a “king” (vertex distance ≤ 2 to all others).
- Very Hard: Sylvester-Gallai theorem (extreme point gives contradiction).

Reduction: Change The Representation

Heuristic: Replace the problem with an equivalent one that is easier to see.

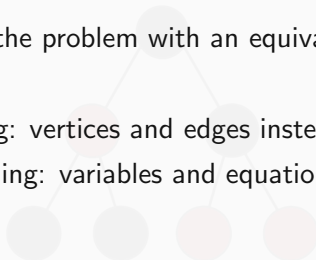
→ Graph modelling: vertices and edges instead of story.



Reduction: Change The Representation

Heuristic: Replace the problem with an equivalent one that is easier to see.

- Graph modelling: vertices and edges instead of story.
- Algebraic encoding: variables and equations instead of logic.



Reduction: Change The Representation

Heuristic: Replace the problem with an equivalent one that is easier to see.

- Graph modelling: vertices and edges instead of story.
- Algebraic encoding: variables and equations instead of logic.
- Modulo arithmetic: finite states instead of infinite.

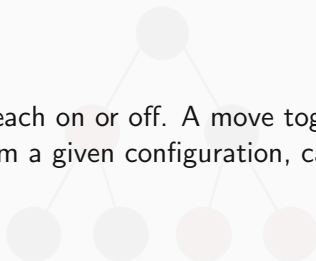
Reduction: Change The Representation

Heuristic: Replace the problem with an equivalent one that is easier to see.

- Graph modelling: vertices and edges instead of story.
- Algebraic encoding: variables and equations instead of logic.
- Modulo arithmetic: finite states instead of infinite.
- This is the bridge to competitive programming.

Worked Example: Reduction – Lamps in a Circle

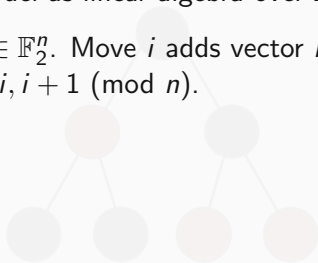
n lamps in a circle, each on or off. A move toggles lamp i and its two neighbours. From a given configuration, can you reach all off?



Reduction: Applying The Concept

How it applies: Model as linear algebra over $\mathbb{F}_2$.

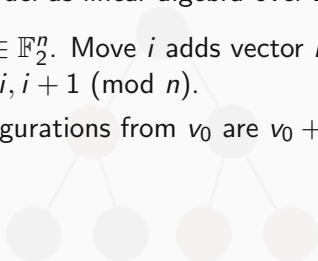
→ State vector $v \in \mathbb{F}_2^n$. Move i adds vector m_i with 1s at positions $i - 1, i, i + 1 \pmod n$.



Reduction: Applying The Concept

How it applies: Model as linear algebra over $\mathbb{F}_2$.

- State vector $v \in \mathbb{F}_2^n$. Move i adds vector m_i with 1s at positions $i-1, i, i+1 \pmod n$.
- Reachable configurations from v_0 are $v_0 + \text{span}\{m_1, \dots, m_n\}$.



Reduction: Applying The Concept

How it applies: Model as linear algebra over $\mathbb{F}_2$.

- State vector $v \in \mathbb{F}_2^n$. Move i adds vector m_i with 1s at positions $i-1, i, i+1 \pmod n$.
- Reachable configurations from v_0 are $v_0 + \text{span}\{m_1, \dots, m_n\}$.
- This is a linear subspace. All-off reachable iff v_0 is in the span.

Reduction: Applying The Concept

How it applies: Model as linear algebra over $\mathbb{F}_2$.

- State vector $v \in \mathbb{F}_2^n$. Move i adds vector m_i with 1s at positions $i-1, i, i+1 \pmod{n}$.
- Reachable configurations from v_0 are $v_0 + \text{span}\{m_1, \dots, m_n\}$.
- This is a linear subspace. All-off reachable iff v_0 is in the span.
- For n not divisible by 3, the moves are independent and span the whole space – always solvable.

Reduction: Applying The Concept

How it applies: Model as linear algebra over $\mathbb{F}_2$.

- State vector $v \in \mathbb{F}_2^n$. Move i adds vector m_i with 1s at positions $i-1, i, i+1 \pmod n$.
- Reachable configurations from v_0 are $v_0 + \text{span}\{m_1, \dots, m_n\}$.
- This is a linear subspace. All-off reachable iff v_0 is in the span.
- For n not divisible by 3, the moves are independent and span the whole space – always solvable.
- For n divisible by 3, the sum of all move vectors is 0, and the invariant is the sum of bits at positions $0, 3, 6, \dots$ modulo 2.

Student Exercises: Reduction

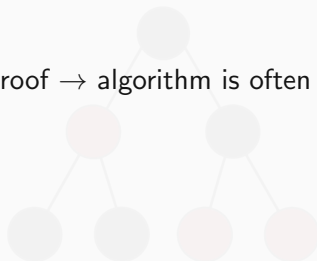
- Basic: Water jugs as graph reachability.
- Easy: Lights Out puzzle on a 3×3 grid.
- Medium: Reduce 15-puzzle parity to permutation sign.
- Hard: Reduction from subset sum to knapsack.
- Very Hard: Prove equivalence of two formulations of the same contest problem (reduction to known NP-complete problem).

From Mathematical Insight To Algorithmic Thinking



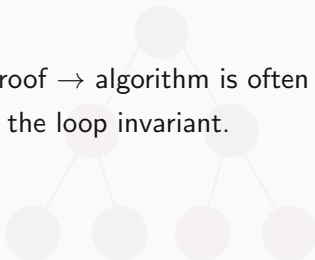
The Bridge To Competitive Programming

→ Mathematical proof → algorithm is often mechanical.

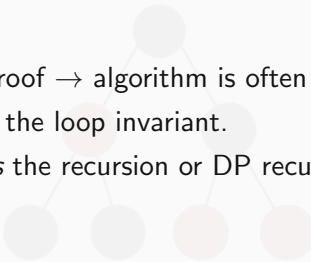


The Bridge To Competitive Programming

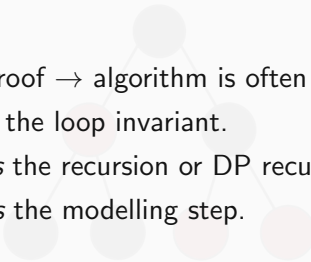
- Mathematical proof → algorithm is often mechanical.
- The invariant *is* the loop invariant.



The Bridge To Competitive Programming

- 
- Mathematical proof → algorithm is often mechanical.
 - The invariant *is* the loop invariant.
 - The induction *is* the recursion or DP recurrence.

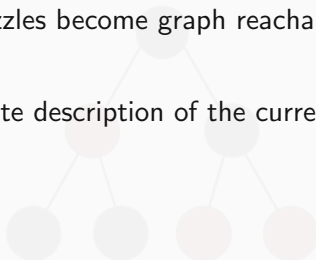
The Bridge To Competitive Programming

- 
- Mathematical proof → algorithm is often mechanical.
 - The invariant *is* the loop invariant.
 - The induction *is* the recursion or DP recurrence.
 - The reduction *is* the modelling step.

Modelling A Puzzle As A Graph (Algorithmic Bridge)

Concept: Many puzzles become graph reachability or shortest path.

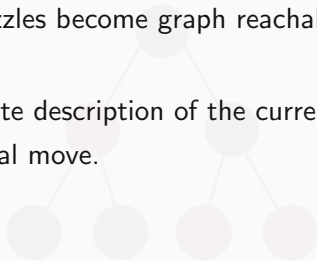
→ State = complete description of the current situation.



Modelling A Puzzle As A Graph (Algorithmic Bridge)

Concept: Many puzzles become graph reachability or shortest path.

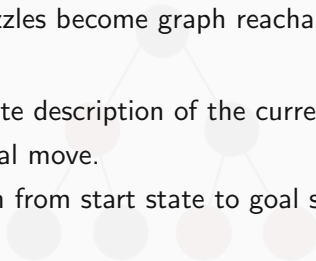
- State = complete description of the current situation.
- Edge = one legal move.



Modelling A Puzzle As A Graph (Algorithmic Bridge)

Concept: Many puzzles become graph reachability or shortest path.

- State = complete description of the current situation.
- Edge = one legal move.
- Solution = path from start state to goal state.



Modelling A Puzzle As A Graph (Algorithmic Bridge)

Concept: Many puzzles become graph reachability or shortest path.

- State = complete description of the current situation.
- Edge = one legal move.
- Solution = path from start state to goal state.
- BFS finds the shortest solution.

Modelling A Puzzle As A Graph (Algorithmic Bridge)

Concept: Many puzzles become graph reachability or shortest path.

- State = complete description of the current situation.
- Edge = one legal move.
- Solution = path from start state to goal state.
- BFS finds the shortest solution.
- An invariant proves two states are in different components.

Worked Example: Water Jugs As A Graph

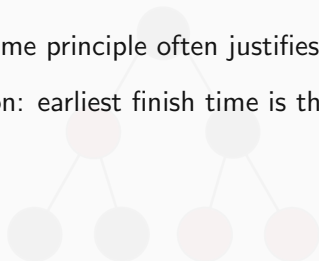
Two jugs of capacities 3 and 5. Target: measure exactly 4 units.
Allowed: fill, empty, pour.

States: (a, b) where $0 \leq a \leq 3$, $0 \leq b \leq 5$. Edges: the six legal operations. BFS from $(0, 0)$ to any state with $a = 4$ or $b = 4$.

Greedy Choices From The Extreme Principle (Algorithmic Bridge)

Concept: The extreme principle often justifies a greedy choice.

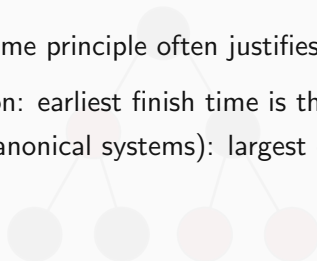
→ Activity selection: earliest finish time is the extreme interval.



Greedy Choices From The Extreme Principle (Algorithmic Bridge)

Concept: The extreme principle often justifies a greedy choice.

- Activity selection: earliest finish time is the extreme interval.
- Coin change (canonical systems): largest coin is the extreme choice.



Greedy Choices From The Extreme Principle (Algorithmic Bridge)

Concept: The extreme principle often justifies a greedy choice.

- Activity selection: earliest finish time is the extreme interval.
- Coin change (canonical systems): largest coin is the extreme choice.
- Fractional knapsack: highest value/weight ratio is the extreme item.

Greedy Choices From The Extreme Principle (Algorithmic Bridge)

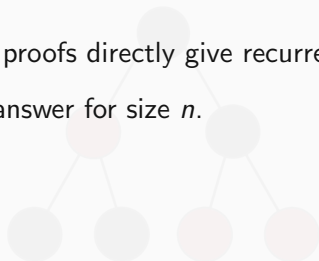
Concept: The extreme principle often justifies a greedy choice.

- Activity selection: earliest finish time is the extreme interval.
- Coin change (canonical systems): largest coin is the extreme choice.
- Fractional knapsack: highest value/weight ratio is the extreme item.
- Proof: exchange argument shows the extreme choice is safe.

Recurrence From Induction (Algorithmic Bridge)

Concept: Inductive proofs directly give recurrences.

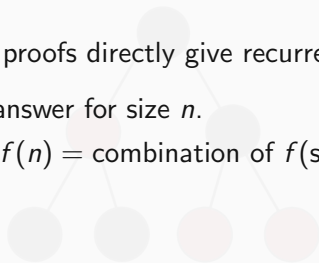
→ Define $f(n)$ = answer for size n .



Recurrence From Induction (Algorithmic Bridge)

Concept: Inductive proofs directly give recurrences.

- Define $f(n)$ = answer for size n .
- Inductive step: $f(n)$ = combination of $f(\text{smaller})$.



Recurrence From Induction (Algorithmic Bridge)

Concept: Inductive proofs directly give recurrences.

- Define $f(n)$ = answer for size n .
- Inductive step: $f(n)$ = combination of $f(\text{smaller})$.
- Memoise or tabulate to avoid recomputation.

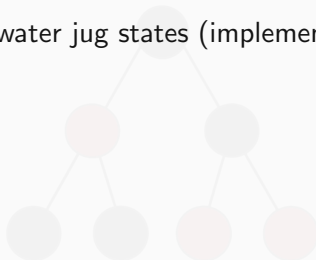
Recurrence From Induction (Algorithmic Bridge)

Concept: Inductive proofs directly give recurrences.

- Define $f(n)$ = answer for size n .
- Inductive step: $f(n)$ = combination of $f(\text{smaller})$.
- Memoise or tabulate to avoid recomputation.
- This is dynamic programming.

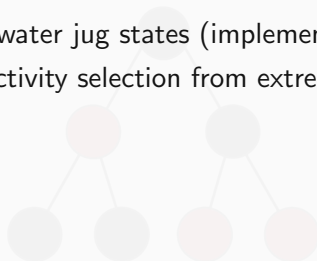
Student Exercises: Algorithmic Bridges

→ Basic: BFS on water jug states (implement conceptually).



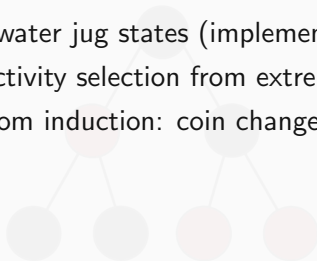
Student Exercises: Algorithmic Bridges

- Basic: BFS on water jug states (implement conceptually).
- Easy: Greedy activity selection from extreme principle.



Student Exercises: Algorithmic Bridges

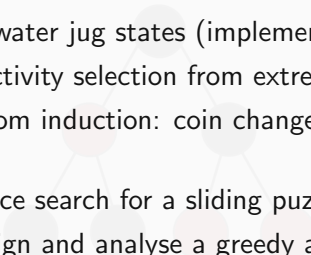
- Basic: BFS on water jug states (implement conceptually).
- Easy: Greedy activity selection from extreme principle.
- Medium: DP from induction: coin change / ways to make sum.



Student Exercises: Algorithmic Bridges

- Basic: BFS on water jug states (implement conceptually).
- Easy: Greedy activity selection from extreme principle.
- Medium: DP from induction: coin change / ways to make sum.
- Hard: State-space search for a sliding puzzle.

Student Exercises: Algorithmic Bridges

- 
- Basic: BFS on water jug states (implement conceptually).
 - Easy: Greedy activity selection from extreme principle.
 - Medium: DP from induction: coin change / ways to make sum.
 - Hard: State-space search for a sliding puzzle.
 - Very Hard: Design and analyse a greedy algorithm for a scheduling problem.

Wrap-Up

Pair 1 complete: Logic, Invariants, Heuristics.

→ Logic tracks knowledge; common knowledge enables coordination.

Algorithmic bridges:



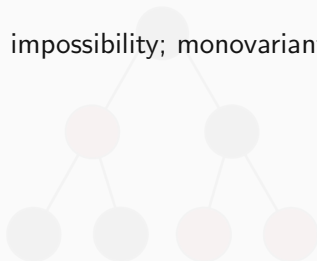
Next: Pair 2 – Core CP Fundamentals: Prefix Sums, Two Pointers, Binary Search.

Wrap-Up

Pair 1 complete: Logic, Invariants, Heuristics.

- Logic tracks knowledge; common knowledge enables coordination.
- Invariants prove impossibility; monovariants prove progress.

Algorithmic bridges:



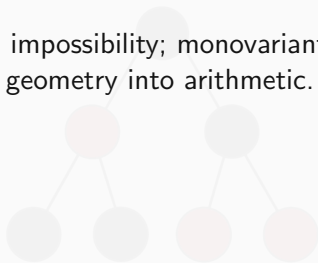
Next: Pair 2 – Core CP Fundamentals: Prefix Sums, Two Pointers, Binary Search.

Wrap-Up

Pair 1 complete: Logic, Invariants, Heuristics.

- Logic tracks knowledge; common knowledge enables coordination.
- Invariants prove impossibility; monovariants prove progress.
- Colouring turns geometry into arithmetic.

Algorithmic bridges:



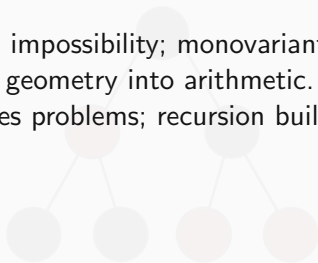
Next: Pair 2 – Core CP Fundamentals: Prefix Sums, Two Pointers, Binary Search.

Wrap-Up

Pair 1 complete: Logic, Invariants, Heuristics.

- Logic tracks knowledge; common knowledge enables coordination.
- Invariants prove impossibility; monovariants prove progress.
- Colouring turns geometry into arithmetic.
- Induction reduces problems; recursion builds algorithms.

Algorithmic bridges:



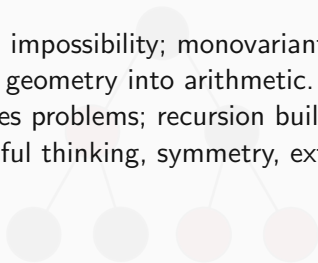
Next: Pair 2 – Core CP Fundamentals: Prefix Sums, Two Pointers, Binary Search.

Wrap-Up

Pair 1 complete: Logic, Invariants, Heuristics.

- Logic tracks knowledge; common knowledge enables coordination.
- Invariants prove impossibility; monovariants prove progress.
- Colouring turns geometry into arithmetic.
- Induction reduces problems; recursion builds algorithms.
- Heuristics: wishful thinking, symmetry, extreme, reduction.

Algorithmic bridges:



Next: Pair 2 – Core CP Fundamentals: Prefix Sums, Two Pointers, Binary Search.

Wrap-Up

Pair 1 complete: Logic, Invariants, Heuristics.

- Logic tracks knowledge; common knowledge enables coordination.
- Invariants prove impossibility; monovariants prove progress.
- Colouring turns geometry into arithmetic.
- Induction reduces problems; recursion builds algorithms.
- Heuristics: wishful thinking, symmetry, extreme, reduction.

Algorithmic bridges:

- State-space modelling → graph algorithms (Pair 3).

Next: Pair 2 – Core CP Fundamentals: Prefix Sums, Two Pointers, Binary Search.

Wrap-Up

Pair 1 complete: Logic, Invariants, Heuristics.

- Logic tracks knowledge; common knowledge enables coordination.
- Invariants prove impossibility; monovariants prove progress.
- Colouring turns geometry into arithmetic.
- Induction reduces problems; recursion builds algorithms.
- Heuristics: wishful thinking, symmetry, extreme, reduction.

Algorithmic bridges:

- State-space modelling → graph algorithms (Pair 3).
- Greedy from extreme principle → greedy algorithms (Pair 2/7).

Next: Pair 2 – Core CP Fundamentals: Prefix Sums, Two Pointers, Binary Search.

Wrap-Up

Pair 1 complete: Logic, Invariants, Heuristics.

- Logic tracks knowledge; common knowledge enables coordination.
- Invariants prove impossibility; monovariants prove progress.
- Colouring turns geometry into arithmetic.
- Induction reduces problems; recursion builds algorithms.
- Heuristics: wishful thinking, symmetry, extreme, reduction.

Algorithmic bridges:

- State-space modelling → graph algorithms (Pair 3).
- Greedy from extreme principle → greedy algorithms (Pair 2/7).
- Induction to recurrence → dynamic programming (Pair 4/8).

Next: Pair 2 – Core CP Fundamentals: Prefix Sums, Two Pointers, Binary Search.